Monitoring-Definition Zeiterfassung

Team f_you_goethe · 26a · SAII · Stand 13.11.2026

Was wir überwachen, wie wir es messen und wann wir reagieren. Ein Messwert ist nur nützlich, wenn er eine Betriebsfrage beantwortet und zu einer Handlung führt: **Betriebsziel → Metrik → PromQL → Dashboard → Alert → Runbook**. Heute läuft auf LM2 die Zeiterfassung mit Prometheus, Node Exporter und Grafana als Docker Compose (alles aus dem App-Repo `SAII_Zeiterfassung`). Später ziehen dieselben Regeln und Dashboards nach Kubernetes um.

Betriebsziele, SLI und SLO

Betriebsziel	SLI (was wir messen)	SLO (Ziel)
Mitarbeitende können zuverlässig stempeln	Anteil erfolgreicher Abfragen der Zeiterfassung durch Prometheus (`up{job="zeiterfassung"}`) an allen Abfragen	≥ 99,5 % pro Kalendermonat (≈ 3,6 h Ausfall erlaubt)
Stempeln ist schnell	Dauer von `POST /stamp`, p95 aus `http_request_duration_seconds`	95 % innerhalb von 500 ms
Die Zeiterfassung arbeitet fehlerfrei	Anteil Antworten mit HTTP 5xx an allen Anfragen	< 1 %
LM2 bleibt nutzbar	RAM-, CPU-Auslastung und freier Platz auf `/` (Node Exporter)	RAM und CPU < 85 %, `/` > 15 % frei

Was wir pro System überwachen

Metriken sind Zahlen über die Zeit («Wie viel? Wie schnell?», z. B. CPU 80 %) und landen in Prometheus/Grafana. Logs sind einzelne Ereignisse mit Kontext («Was genau ist passiert?», z. B. «Failed SSH login from …») und landen später in ELK oder Loki.

System	Metriken (Quelle)	Logs (Quelle)	Stand
LM2 (Host)	CPU, RAM, Dateisystem, Netzwerk, Erreichbarkeit (Node Exporter :9100)	journald/syslog, `auth.log` (SSH-Logins, Fehlversuche), Docker-Daemon	Metriken **umgesetzt** Logs **geplant**
Zeiterfassung (App)	Erreichbarkeit, Anfragen pro Sekunde, Fehlerquote 5xx, Antwortzeit p95 je Route (`/metrics` :8080)	Access-Log auf stdout (heute). Ziel: JSON-Logs mit Zeit, Person, Action, Pfad, Status, Dauer, Trace-ID; Login-Fehlversuche	Metriken **umgesetzt** Logs **geplant**
Container	CPU und RAM pro Container, Neustarts (cAdvisor)	stdout/stderr der Container	**geplant**
Datenbank (PostgreSQL)	Erreichbarkeit, Verbindungen, Grösse, Abfragedauer (postgres_exporter)	Fehler, langsame Abfragen	**geplant**
Kubernetes, Argo CD (Ziel)	Pods bereit, Neustarts (kube-state-metrics), Sync- und Health-Status von Argo CD	Kubernetes-Events, Sync-Logs	**geplant**
Monitoring selbst	`up` von Prometheus, Regelauswertung, Grafana-Health	Container-Logs	**umgesetzt**
Kali (KL1, KL2)	Kali wird nicht selbst überwacht. Es ist unser Test- und Angreifersystem. Seine Tests sollen **auf LM2 sichtbar** werden: Anfragespitze und 4xx-Häufung in den App-Metriken	Fehlgeschlagene SSH-Logins in `auth.log`, 404-/Login-Fehlerserien im App-Log, Nmap-Scans → Alert «viele fehlgeschlagene Logins»	**geplant**
LM1, CI	Build-Status in GitHub Actions; LM1 optional mit Node Exporter	CI-Logs in GitHub	**geplant**
Firewall LF1	nicht unter unserer Kontrolle (Dozent)	IPFire-Logs bei Bedarf beim Dozenten	–
Mattermost	Fehlversuche beim Senden von Alerts (Alertmanager)	–	**geplant**

Dashboard «LM2 und Zeiterfassung» (Grafana :3000)

Jedes Panel beantwortet eine Frage, mit passender Einheit und Zeitraum (Standard: letzte 15 Minuten, Aktualisierung alle 10 s).

Panel	Frage	PromQL (gekürzt)
RAM-Auslastung, -Verlauf	Hat LM2 genug Arbeitsspeicher, seit wann steigt er?	`100 * (1 - MemAvailable / MemTotal)`
CPU-Auslastung, -Verlauf	Ist die einzige vCPU von LM2 überlastet?	`100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))`
Root-Dateisystem frei	Läuft `/` voll (Daten von App, Prometheus, Docker)?	`100 * avail_bytes / size_bytes` (mountpoint `/`)

Panel	Frage	PromQL (gekürzt)
Target LM2, Target Zeiterfassung	Liefern Host und App Messwerte (UP/DOWN)?	`up{job="lm2-node"}`, `up{job="zeiterfassung"}`
Anfragen pro Sekunde	Wie stark wird die Zeiterfassung genutzt, auf welchen Routen?	`sum by (endpoint) (rate(http_requests_total[5m]))`
Fehlerquote 5xx	Wie viele Anfragen scheitern serverseitig?	`5xx-Rate / Gesamtrate * 100`
Stempeln p95-Latenz	Halten wir das Latenz-SLO von 500 ms ein?	`histogram_quantile(0.95, … endpoint="/stamp")`

Alerts

Jeder Alert hat eine Bedingung, eine Wartezeit gegen kurze Spitzen, Kontext (Severity, Team `platform`, Host) und ein Runbook. Lebenszyklus: Normal → Pending (Wartezeit läuft) → Firing → Resolved.

Alert	Bedingung	Wartezeit	Severity
`LM2NodeDown`	Node Exporter auf LM2 nicht erreichbar	1 min	critical
`LM2HighMemoryUsage`	RAM-Auslastung > 85 %	5 min	warning
`LM2HighCpuUsage`	CPU-Auslastung > 85 %	5 min	warning
`LM2RootFilesystemLow`	weniger als 15 % frei auf `/`	10 min	warning
`ZeiterfassungDown`	`/metrics` der Zeiterfassung nicht erreichbar	1 min	critical
`ZeiterfassungHighErrorRate`	mehr als 5 % der Anfragen mit HTTP 5xx	5 min	warning
`ZeiterfassungSlowStamps`	p95 von `POST /stamp` über 500 ms (SLO verletzt)	10 min	warning

Die Runbooks liegen im App-Repo unter `monitoring/runbooks/` und sind in jedem Alert verlinkt (separates Dokument «Runbooks»).

Offene Punkte

- **Alertmanager → Mattermost** fehlt noch: Alerts sind heute nur in Prometheus unter `/alerts` sichtbar.
- **Logs**: ELK oder Loki. LM2 hat nur 1 vCPU, Elasticsearch ist dafür schwer. Empfehlung: Loki, falls der Dozent es zulässt.
- **Zugriff**: Ports 8080, 9090, 9100 und 3000 sind heute direkt aus lan0 erreichbar (geprüft 13.11.). Ziel: nur HTTPS 443 über den Ingress, Rest clusterintern.
- **Grafana-Passwort** `saii-demo` gilt nur für die Schulungsumgebung und wird vor der Übergabe ersetzt.
- **Umzug nach Kubernetes**: dieselben Regeln als PrometheusRule, Dashboard als ConfigMap, verteilt durch Argo CD.